

LAPORAN PRAKTIKUM PEMROGRAMAN BERORIENTASI OBJEK
FUNGSI CRUD DENGAN DATABASE MySQL



OLEH :
TEGUH ESA MAULANNA
2411532005

DOSEN PENGAMPU:
NURFIAH, S.ST, M.Kom.,

FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS
PADANG 2025

A. Pendahuluan

MySQL merupakan sebuah manajemen basis data yang menggunakan perintah dasar SQL (Structured Query Language). SQL merupakan suatu sintak perintah tertentu yang digunakan untuk mengelola suatu database. SQL merupakan bahasa standar yang digunakan dalam manajemen basis data relasional atau disebut sebagai Relational Database Management System (RDBMS).

MySQL merupakan sebuah driver atau library yang digunakan untuk dengan sistem manajemen MySQL seperti Java, python, PHP, C++, dan .NET. Fungsinya adalah sebagai perantara untuk komunikasi antara aplikasi dan database, sehingga operasi seperti membaca, menulis, dan memodifikasi dapat dilakukan dengan lebih praktis dan efisien.

DAO atau Data Access Object adalah sebuah cara untuk mengatur kode program agar dapat menangani komunikasi antara aplikasi dan basis data. DAO berisikan objek yang menyediakan abstract interface untuk beberapa method yang berhubungan dengan database seperti mengambil data atau membaca data (read), menyimpan data(create), menghapus data (delete), mengubah data(update).

CRUD merupakan singkatan dari Create, Read, Update, dan Delete merupakan sebuah operasi dasar dalam pengelolaan data dalam sebuah database terkhusus pada database yang bersifat rasional. CRUD merupakan pondasi dasar dalam pengembangan sistem atau aplikasi yang berhubungan dengan pengolahan data.

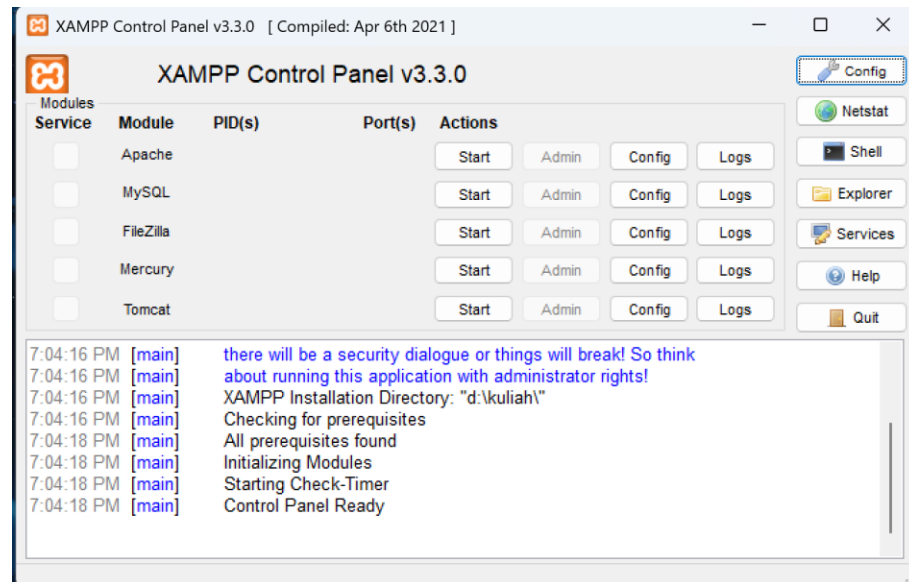
B. Tujuan

1. Memahami konsep dasar MySQL sebagai sistem manajemen basis data.
2. Mempelajari fungsi driver atau library MySQL Connector sebagai penghubung antara aplikasi dan database.
3. Mengimplementasikan operasi dasar CRUD pada pemrograman berbasis objek.

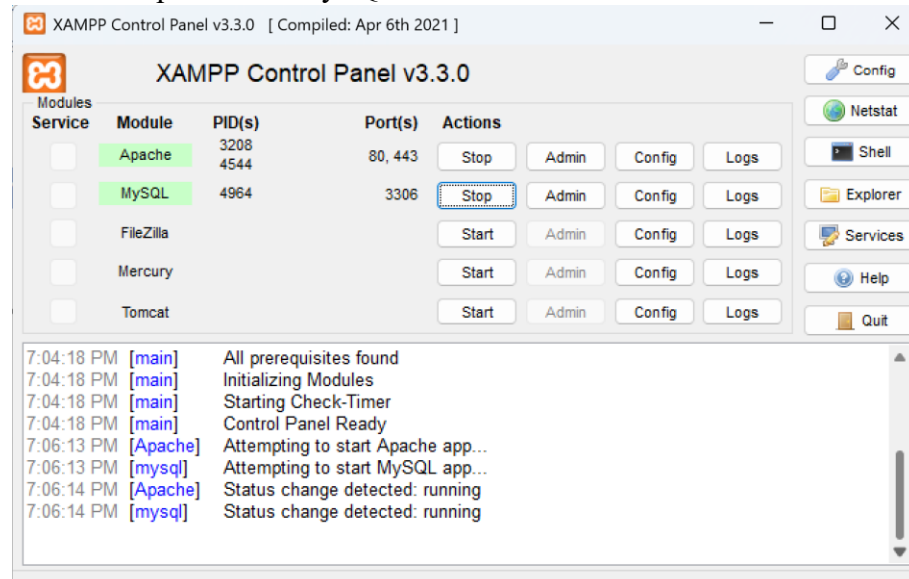
C. Langkah Kerja Praktikum

a) Jalankan XAMPP

1. Buka atau install aplikasi XAMPP.

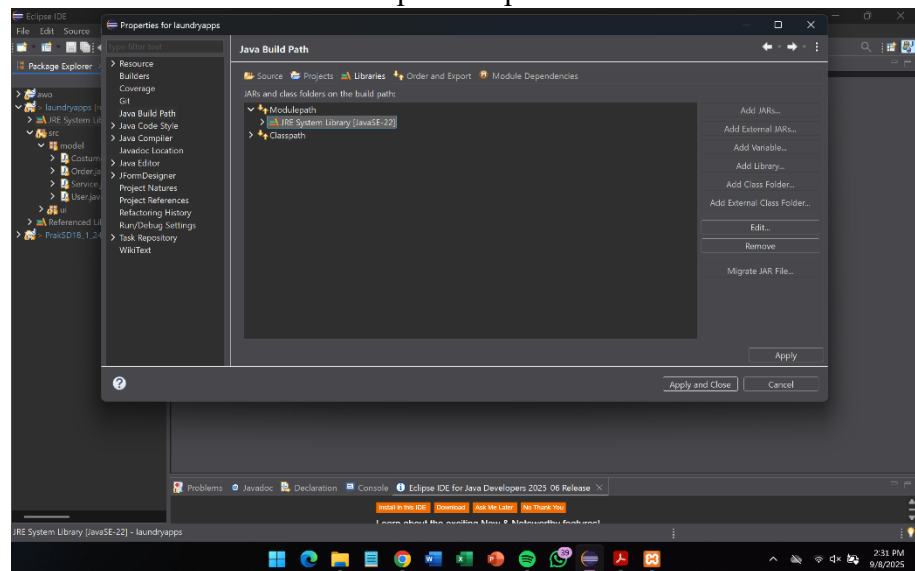
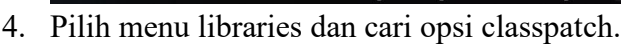


2. Aktifkan Apache dan MySQL.

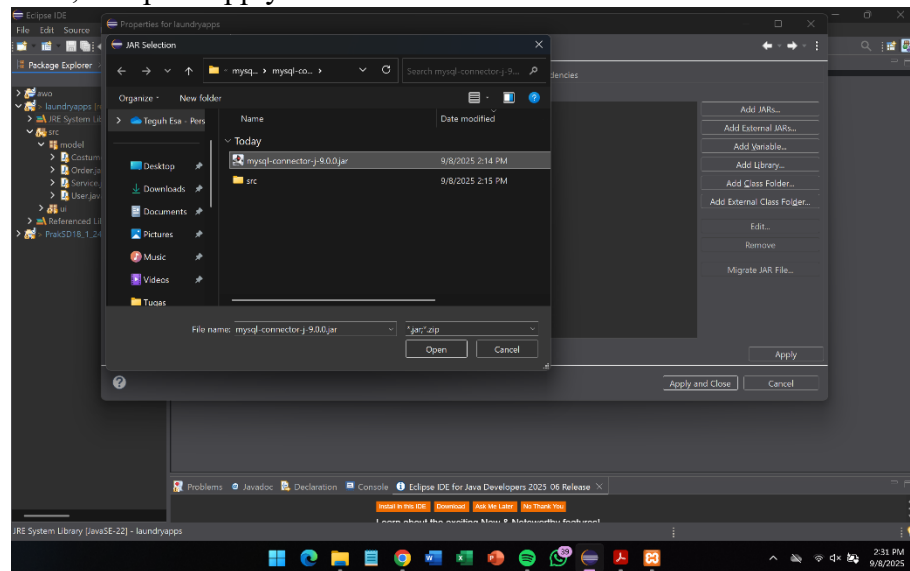


b) Tambahkan driver MySQL Connector

1. Install driver MySQL Connector.
2. Buka aplikasi eclipse untuk menambahkan driver tersebut.
3. Buka project yang akan digunakan, cari menu JRE System Library, klik kanan pada menu tersebut, pilih opsi Build Path lalu Configure Build Path.

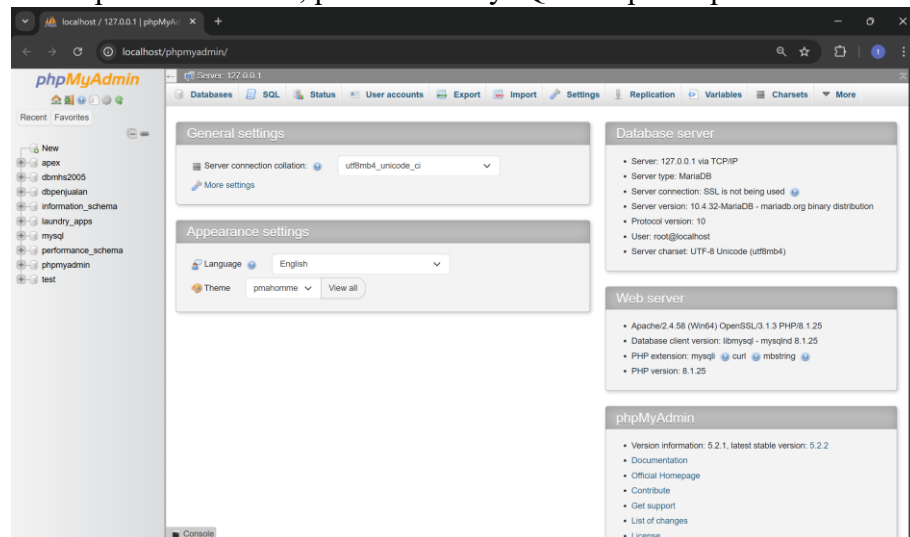


5. Tambahkan file MySQL connector dengan memilih opsi Add External JaRs, lalu pilih apply dan close.

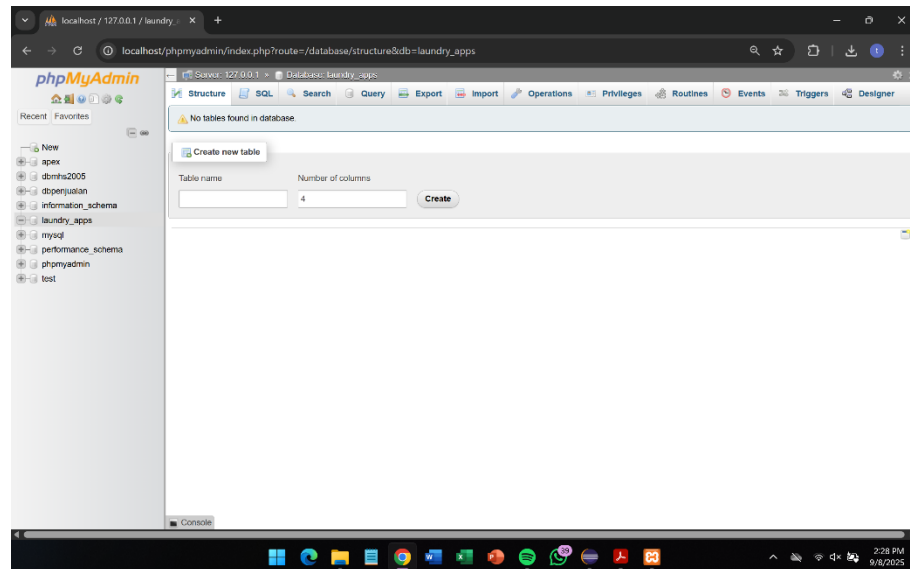


- c) Membuat sebuah database dan tabel.

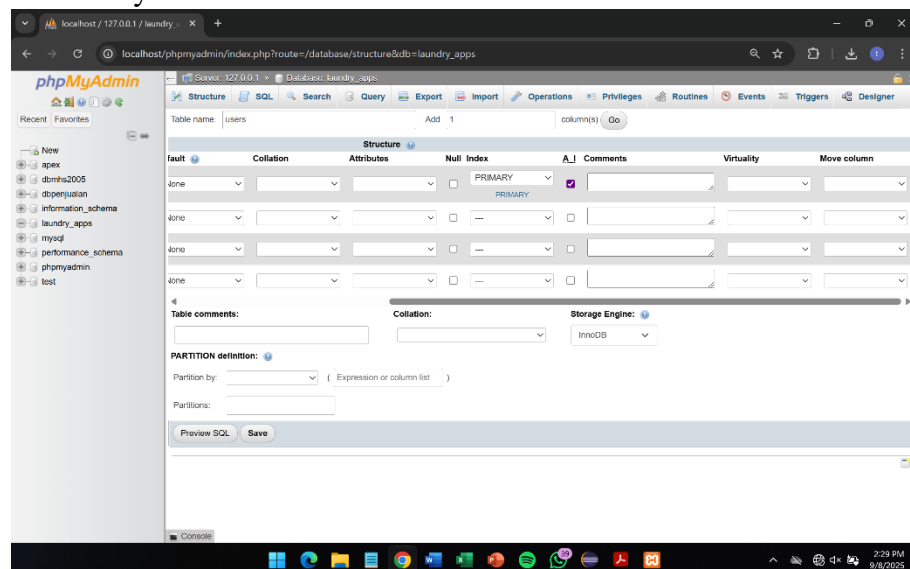
1. Pada aplikasi XAMPP, pilih menu MySQL dan pilih opsi admin.



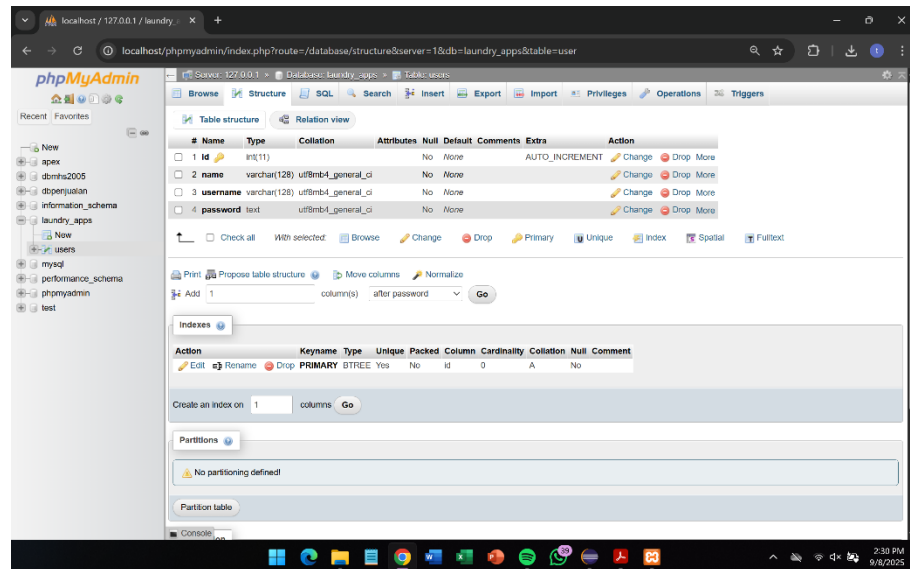
2. Buat sebuah database baru dengan klik opsi new pada bagian kiri layar. Database tersebut diberi nama laundry_apps.



3. Buat tabel baru dengan nama user didalam database yang sudah dibuat sebelumnya.



Tabel tersebut diberi nama user yang berisikan 4 buah kolom, yaitu id (int dan primary key), name (varchar), username (varchar), dan password (text).



d) Membuat koneksi ke database MySQL.

1. Buat sebuah package baru didalam project laundryapps dengan nama config.
2. Buat class baru dengan nama database.
3. Import java.sql.*

```
import java.sql.*;
```

Mengimpor semua class dari package java.sql yang digunakan untuk koneksi database.

4. Membuat method untuk koneksi database.

```
public static Connection koneksi() {
    try {
        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection conn = DriverManager.getConnection("jdbc:mysql://localhost/laundry_apps",
            "root", "");
        return conn;
    } catch (Exception e) {
        JOptionPane.showMessageDialog(null, e);
        return null;
    }
}
```

Method ini bertipe static, method akan mengkoneksikan database dengan memanggil driver JDBC MySQL. Memanggil database dari database jdbc:mysql://localhost/laundry_apps dengan username "root" dan password "". Jika koneksi berhasil method mengembalikan objek conn. Jika terdapat error, program akan menampilkan pesan error.

5. Membuat sebuah class khusus untuk melakukan pengujian koneksi.

```

package config;

import java.sql.*;

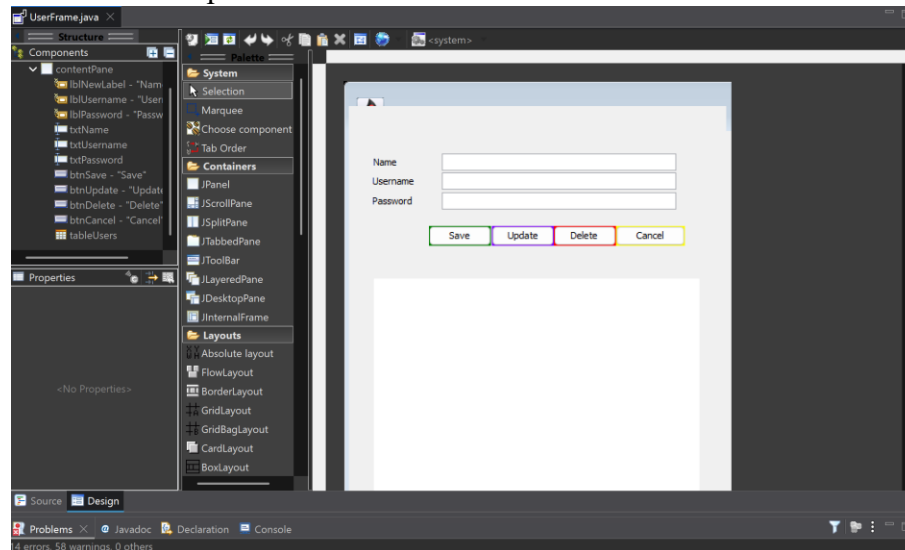
public class tes {
    public static void main(String[] args) {
        Connection conn = Database.koneksi();
        if (conn != null) {
            JOptionPane.showMessageDialog(null, "Koneksi ke database berhasil!");
        } else {
            JOptionPane.showMessageDialog(null, "Koneksi ke database gagal!");
        }
    }
}

```

Pengecekan koneksi dilakukan dengan memanggil method koneksi() dari class database. Jika koneksi berhasil variable conn berisi objek connection, jika gagal variable conn nilainya null.

e) Membuat tampilan CRUD

1. Buatlah sebuah class GUI menggunakan JFrame dengan nama UserFrame dalam package ui.
2. Mendesain tampilan dari class UserFrame tersebut.



Terdiri dari 3 JTextField yaitu txtName untuk mengisikan nama pengguna, txtUsername untuk mengisikan username pengguna, dan txtPassword untuk mengisikan password pengguna. Terdapat 4 JButton yaitu btnSave untuk menyimpan input dari pengguna, btnUpdate untuk melakukan perubahan, btnDelete untuk menghapus, dan btnCancel untuk membatalkan proses atau perubahan dari pengguna.

f) Membuat tabel model.

1. Buat package baru didalam project laundryapps dengan nama table.
2. Buat sebuah class didalam package tersebut dengan nama TableUser.
3. Deklarasikan class dan konstruktor.


```
public class TableUser extends AbstractTableModel {
    List<User> ls;
    private String[] columnNames = {"ID", "Name", "Username", "Password"};
    public TableUser(List<User> ls) {
        this.ls = ls;
    }
}
```

TableUser merupakan sebuah subclass dari AbstractTableModel. Class akan menyimpan daftar objek user dalam bentuk array dengan nama columnNames berisikan daftar kolom dari tabel yaitu (ID, Name, Username, dan Password). Konstruktor dari class ini akan menerima parameter berupa List<User>ls yang nantinya data user ini akan digunakan sebagai isi dari tabel.

4. Menampilkan jumlah tabel.

```
@Override
public int getRowCount() {
    // TODO Auto-generated method stub
    return ls.size();
}
```

Menggunakan method override untuk mengembalikan jumlah baris, jumlah baris merupakan nilai jumlah data user dalam list.

5. Menampilkan jumlah kolom.

```
@Override
public int getColumnCount() {
    // TODO Auto-generated method stub
    return 4;
}
```

Menggunakan method override untuk mengembalikan jumlah kolom, jumlah kolom selalu bernilai 4 yaitu (ID, Name, Username, dan Password) jadi kembalikan nilai 4.

6. Menampilkan nama kolom.

```
@Override
public String getColumnName(int column) {
    // TODO Auto-generated method stub
    return columnNames[column];
}
```

Menggunakan method override untuk mengembalikan nilai nama – nama kolom yang ada sesuai indeks.

7. Menampilkan isi tabel.

```

@Override
public Object getValueAt(int rowIndex, int columnIndex) {
    // TODO Auto-generated method stub
    switch (columnIndex) {
        case 0:
            return ls.get(rowIndex).getId();
        case 1:
            return ls.get(rowIndex).getNama();
        case 2:
            return ls.get(rowIndex).getUsername();
        case 3:
            return ls.get(rowIndex).getPassword();
        default:
            return null;
    }
}

```

Menggunakan method override untuk mengambil isi tabel. Method akan menggunakan method `rowIndex` untuk mengambil indeks baris ke berapa dan `columnIndex` untuk mengambil indeks kolom ke berapa, lalu mengambil data dari user sesuai dengan baris, lalu menampilkan atribut sesuai dengan kolom.

g) Membuat fungsi DAO

1. Buatlah package baru didalam project laundryapps dengan nama DAO.
2. Buatlah class dengan tipe interface dengan nama UserDao didalam package tersebut.
3. Membuatkan method atau fungsi untuk CRUD.

```

public interface UserDao {
    void save(User user);
    public List<User> show();
    public void delete(String id);
    public void update(User user);
}

```

Terdiri dari beberapa operasi CRUD yaitu `save` untuk menyimpan segala proses operasi yang dilakukan dan menyimpannya dalam objek user, `show` untuk mengambil dan menampilkan semua data dari user dalam bentuk list, `delete` untuk menghapus atribut user berdasarkan id, dan `update` untuk mengubah data user dengan data yang baru.

h) Menggunakan fungsi DAO

1. Buatlah class baru didalam package DAO dengan nama UserRepo, class ini merupakan implementasi dari interface UserDao.
2. Deklarasikan class dan query SQL.

```
public class UserRepo implements UserDao {
    private Connection connection;
    final String insert = "INSERT INTO user (name, username, password) VALUES (?, ?, ?);";
    final String select = "SELECT * FROM user;";
    final String delete = "DELETE FROM user WHERE id=?;";
    final String update = "UPDATE user SET name=?, username=?, password=? WHERE id=?;";
}
```

- Koneksikan class ini ke database menggunakan objek connection. Simpan query SQL sebagai sebuah konstanta yaitu :
- Insert menggunakan query INSERT INTO, untuk menambahkan data.
- Select menggunakan query SELECT * FROM untuk menampilkan data.
- Delete menggunakan query DELETE FROM untuk menghapus data.
- Update menggunakan query UPDATE untuk melakukan perubahan pada data.

3. Tambahkan konstruktor class.

```
public UserRepo() {
    connection = Database.koneksi();
}
```

Konstruktor ini akan dipanggil saat objek UserRepo dibuat lalu memanggil method koneksi dari class database agar connection bisa digunakan.

4. Buat method untuk save

```
@Override
public void save(User user) {
    PreparedStatement st = null;
    try {
        st = connection.prepareStatement(insert);
        st.setString(1, user.getNama());
        st.setString(2, user.getUsername());
        st.setString(3, user.getPassword());
        st.executeUpdate();
    } catch (SQLException e) {
        e.printStackTrace();
    } finally {
        try {
            st.close();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Method ini membuat syntax SQL berdasarkan query INSERT dan diisi sesuai index yaitu 1 untuk nama, 2 untuk username, dan 3 untuk password. Ketika query dijalankan program akan menambahkan baris data ke dalam database.

5. Buat method untuk show

```
@Override
public List<User> show() {
    List<User> ls=null;
    try {
        ls = new ArrayList<User>();
        Statement st = connection.createStatement();
        ResultSet rs = st.executeQuery(select);
        while(rs.next()) {
            User user = new User();
            user.setId(rs.getString("id"));
            user.setNama(rs.getString("name"));
            user.setUsername(rs.getString("username"));
            user.setPassword(rs.getString("password"));
            ls.add(user);
        }
    } catch(SQLException e) {
        Logger.getLogger(UserDao.class.getName()).log(Level.SEVERE, null, e);
    } return ls;
}
```

Method akan membuat sebuah arraylist untuk menyimpan data, dan membuat syntax SQL berdasarkan query SELECT. Ketika query dijalankan hasilnya disimpan ke ResultSet lalu memindahkan setiap baris hasil query. Baris tersebut berisikan sebuah objek user baru dengan mengambil data dari kolom id, name, username, dan password lalu menambahkannya ke list ls, dan menampilkan list tersebut.

6. Buat method untuk update

```
@Override
public void update(User user) {
    PreparedStatement st = null;
    try {
        st = connection.prepareStatement(update);
        st.setString(1, user.getNama());
        st.setString(2, user.getUsername());
        st.setString(3, user.getPassword());
        st.setString(4, user.getId());
        st.executeUpdate();
    } catch(SQLException e) {
        e.printStackTrace();
    } finally {
        try {
            st.close();
        } catch(SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Method ini membuat syntax SQL berdasarkan query UPDATE. Query akan disusun berdasarkan indeks yaitu 2 untuk name, 2 untuk username, 3 untuk password, dan 4 untuk WHERE.

7. Buat method untuk delete

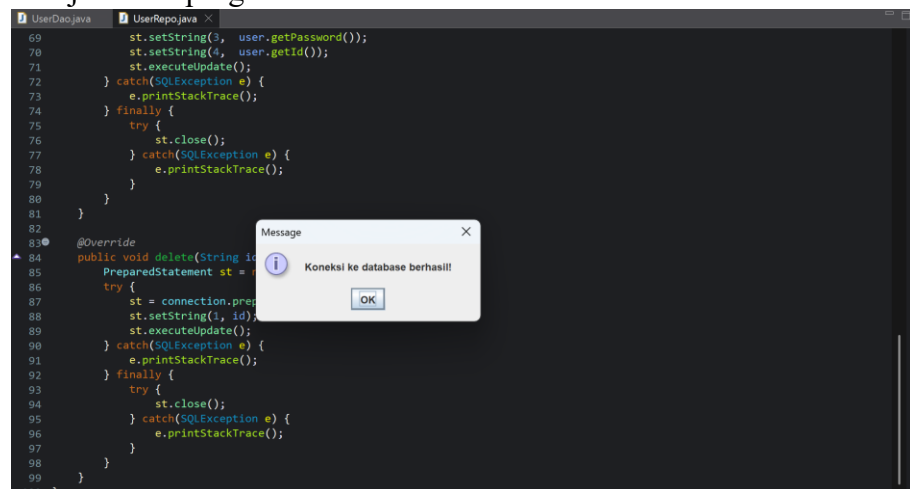
```

@Override
public void delete(String id) {
    PreparedStatement st = null;
    try {
        st = connection.prepareStatement(delete);
        st.setString(1, id);
        st.executeUpdate();
    } catch (SQLException e) {
        e.printStackTrace();
    } finally {
        try {
            st.close();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
}

```

Method ini membuat syntax SQL berdasarkan query DELETE. Query akan mengambil id dari user lalu menjalankan query.

8. Menjalankan program.



The screenshot shows an IDE with two tabs: 'UserDao.java' and 'UserRepo.java'. The 'UserRepo.java' tab is active, displaying the following code:

```

69 st.setString(3, user.getPassword());
70 st.setString(4, user.getId());
71 st.executeUpdate();
72 } catch (SQLException e) {
73     e.printStackTrace();
74 } finally {
75     try {
76         st.close();
77     } catch (SQLException e) {
78         e.printStackTrace();
79     }
80 }
81 }
82
83 @Override
84 public void delete(String id) {
85     PreparedStatement st = null;
86     try {
87         st = connection.prepareStatement(delete);
88         st.setString(1, id);
89         st.executeUpdate();
90     } catch (SQLException e) {
91         e.printStackTrace();
92     } finally {
93         try {
94             st.close();
95         } catch (SQLException e) {
96             e.printStackTrace();
97         }
98     }
99 }
100 }

```

A message dialog box is overlaid on the code, titled 'Message', with the text 'Koneksi ke database berhasil!' (Database connection successful!) and an 'OK' button.

D. Kesimpulan

Dari praktikum ini kita telah mempelajari bahwa MySQL merupakan sistem manajemen basis data relasional yang menggunakan SQL sebagai bahasa standar pengolahan data. MySQL connector digunakan sebagai penghubung antara aplikasi pemrograman dengan sistem basis data. Dengan menerapkan pola desain DAO, proses pengelolaan data dalam aplikasi menjadi lebih terstruktur karena logika akses data dipisahkan dari logika utama program. Selain itu, implementasi operasi CRUD sangat penting sebagai pemahaman dasar dalam mengelola data, karena menjadi pondasi bagi hampir semua aplikasi berbasis database.